EDA_exercício

March 16, 2026

1 Aula 4 — EDA Profissional (Versão Avançada) - Para Aula

Dataset: Airbnb NYC

1.1 Objetivos Desta Aula

- **Explorar um dataset real de mercado:** Entender a estrutura e o conteúdo de dados reais.
- **Detectar e tratar valores faltantes e outliers:** Aprender técnicas para limpar e preparar os dados.
- **Realizar análise por categoria e por localização:** Investigar padrões e tendências em diferentes segmentos e regiões.
- **Construir gráficos profissionais e informativos:** Utilizar bibliotecas avançadas para visualizações de alta qualidade.
- **Introduzir correlação entre variáveis:** Entender as relações entre diferentes atributos do dataset.
- **Engenharia de Features:** Criar novas variáveis para enriquecer a análise.
- **Análise Geoespacial:** Visualizar dados em um contexto geográfico.
- **Interpretar e comunicar insights:** Extrair conclusões significativas dos dados.

1.2 1) Importação das bibliotecas Essenciais

Nesta seção, importamos todas as bibliotecas Python necessárias para a Análise Exploratória de Dados (EDA).

- `pandas`: Para manipulação e análise de dados em formato de DataFrame.
- `matplotlib.pyplot`: Para a criação de gráficos estáticos.
- `seaborn`: Baseado no matplotlib, oferece uma interface de alto nível para criar gráficos estatísticos atraentes e informativos.
- `numpy`: Para operações numéricas e matemáticas, especialmente útil para lidar com arrays e cálculos estatísticos.
- `folium`: Para criar mapas interativos e visualizar dados geoespaciais.

[124]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import folium

pd.set_option("display.max_columns", None)
```

```
pd.set_option("display.width", None)
```

1.3 2) Carregar o Dataset

Carregamos o dataset do Airbnb NYC a partir de um arquivo CSV. Este é o primeiro passo para qualquer análise de dados, onde os dados brutos são lidos e transformados em um DataFrame do pandas para fácil manipulação.

[125]:
```
df = pd.read_csv('AB_NYC_2019.csv')
print(df.head())
```

```
     id                                              name  host_id  \
0  2539                Clean & quiet apt home by the park     2787
1  2595                             Skylit Midtown Castle     2845
2  3647               THE VILLAGE OF HARLEM…NEW YORK !     4632
3  3831                    Cozy Entire Floor of Brownstone     4869
4  5022  Entire Apt: Spacious Studio/Loft by central park     7192

     host_name neighbourhood_group neighbourhood  latitude  longitude  \
0         John            Brooklyn    Kensington  40.64749  -73.97237
1     Jennifer           Manhattan       Midtown  40.75362  -73.98377
2    Elisabeth           Manhattan        Harlem  40.80902  -73.94190
3  LisaRoxanne            Brooklyn  Clinton Hill  40.68514  -73.95976
4        Laura           Manhattan   East Harlem  40.79851  -73.94399

         room_type  price  minimum_nights  number_of_reviews last_review  \
0     Private room    149               1                  9  2018-10-19
1  Entire home/apt    225               1                 45  2019-05-21
2     Private room    150               3                  0         NaN
3  Entire home/apt     89               1                270  2019-07-05
4  Entire home/apt     80              10                  9  2018-11-19

   reviews_per_month  calculated_host_listings_count  availability_365
0               0.21                               6               365
1               0.38                               2               355
2                NaN                               1               365
3               4.64                               1               194
4               0.10                               1                 0
```

1.4 3) Visão Geral e Estrutura dos Dados

Esta seção fornece uma visão geral rápida do dataset, incluindo suas dimensões, tipos de dados e um resumo estatístico. Isso nos ajuda a entender a estrutura dos dados e identificar possíveis problemas, como valores ausentes ou tipos de dados incorretos.

[126]:
```
# Exiba o número de linhas e colunas do DataFrame (df.shape)
print("(Linhas, Colunas) =", end = " ")
print(df.shape)
```

```python
print("\n")

# Exiba informações sobre o DataFrame, incluindo tipos de dados e valores␣
↪não-nulos (df.info())
print("Informações gerais:")
df.info()
print("\n")

# Exiba estatísticas descritivas para as colunas numéricas (df.describe())
print("Estatísticas numéricas:")
print(df.describe())
print("\n")

# Exiba estatísticas descritivas para todas as colunas (incluindo categóricas)␣
↪(df.describe(include="all"))
print("Estatísticas gerais:")
print(df.describe(include = "all"))
```

```
(Linhas, Colunas) = (48895, 16)


Informações gerais:
<class 'pandas.DataFrame'>
RangeIndex: 48895 entries, 0 to 48894
Data columns (total 16 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   id                              48895 non-null  int64
 1   name                            48879 non-null  str
 2   host_id                         48895 non-null  int64
 3   host_name                       48874 non-null  str
 4   neighbourhood_group             48895 non-null  str
 5   neighbourhood                   48895 non-null  str
 6   latitude                        48895 non-null  float64
 7   longitude                       48895 non-null  float64
 8   room_type                       48895 non-null  str
 9   price                           48895 non-null  int64
 10  minimum_nights                  48895 non-null  int64
 11  number_of_reviews               48895 non-null  int64
 12  last_review                     38843 non-null  str
 13  reviews_per_month               38843 non-null  float64
 14  calculated_host_listings_count  48895 non-null  int64
 15  availability_365                48895 non-null  int64
dtypes: float64(3), int64(7), str(6)
memory usage: 6.0 MB
```

Estatísticas numéricas:

	id	host_id	latitude	longitude	price \
count	4.889500e+04	4.889500e+04	48895.000000	48895.000000	48895.000000
mean	1.901714e+07	6.762001e+07	40.728949	-73.952170	152.720687
std	1.098311e+07	7.861097e+07	0.054530	0.046157	240.154170
min	2.539000e+03	2.438000e+03	40.499790	-74.244420	0.000000
25%	9.471945e+06	7.822033e+06	40.690100	-73.983070	69.000000
50%	1.967728e+07	3.079382e+07	40.723070	-73.955680	106.000000
75%	2.915218e+07	1.074344e+08	40.763115	-73.936275	175.000000
max	3.648724e+07	2.743213e+08	40.913060	-73.712990	10000.000000

	minimum_nights	number_of_reviews	reviews_per_month \
count	48895.000000	48895.000000	38843.000000
mean	7.029962	23.274466	1.373221
std	20.510550	44.550582	1.680442
min	1.000000	0.000000	0.010000
25%	1.000000	1.000000	0.190000
50%	3.000000	5.000000	0.720000
75%	5.000000	24.000000	2.020000
max	1250.000000	629.000000	58.500000

	calculated_host_listings_count	availability_365
count	48895.000000	48895.000000
mean	7.143982	112.781327
std	32.952519	131.622289
min	1.000000	0.000000
25%	1.000000	0.000000
50%	1.000000	45.000000
75%	2.000000	227.000000
max	327.000000	365.000000

Estatísticas gerais:

	id	name	host_id	host_name \
count	4.889500e+04	48879	4.889500e+04	48874
unique	NaN	47905	NaN	11452
top	NaN	Hillside Hotel	NaN	Michael
freq	NaN	18	NaN	417
mean	1.901714e+07	NaN	6.762001e+07	NaN
std	1.098311e+07	NaN	7.861097e+07	NaN
min	2.539000e+03	NaN	2.438000e+03	NaN
25%	9.471945e+06	NaN	7.822033e+06	NaN
50%	1.967728e+07	NaN	3.079382e+07	NaN
75%	2.915218e+07	NaN	1.074344e+08	NaN
max	3.648724e+07	NaN	2.743213e+08	NaN

	neighbourhood_group	neighbourhood	latitude	longitude \
count	48895	48895	48895.000000	48895.000000

unique	5	221	NaN	NaN
top	Manhattan	Williamsburg	NaN	NaN
freq	21661	3920	NaN	NaN
mean	NaN	NaN	40.728949	-73.952170
std	NaN	NaN	0.054530	0.046157
min	NaN	NaN	40.499790	-74.244420
25%	NaN	NaN	40.690100	-73.983070
50%	NaN	NaN	40.723070	-73.955680
75%	NaN	NaN	40.763115	-73.936275
max	NaN	NaN	40.913060	-73.712990

	room_type	price	minimum_nights	number_of_reviews	\
count	48895	48895.000000	48895.000000	48895.000000	
unique	3	NaN	NaN	NaN	
top	Entire home/apt	NaN	NaN	NaN	
freq	25409	NaN	NaN	NaN	
mean	NaN	152.720687	7.029962	23.274466	
std	NaN	240.154170	20.510550	44.550582	
min	NaN	0.000000	1.000000	0.000000	
25%	NaN	69.000000	1.000000	1.000000	
50%	NaN	106.000000	3.000000	5.000000	
75%	NaN	175.000000	5.000000	24.000000	
max	NaN	10000.000000	1250.000000	629.000000	

	last_review	reviews_per_month	calculated_host_listings_count	\
count	38843	38843.000000	48895.000000	
unique	1764	NaN	NaN	
top	2019-06-23	NaN	NaN	
freq	1413	NaN	NaN	
mean	NaN	1.373221	7.143982	
std	NaN	1.680442	32.952519	
min	NaN	0.010000	1.000000	
25%	NaN	0.190000	1.000000	
50%	NaN	0.720000	1.000000	
75%	NaN	2.020000	2.000000	
max	NaN	58.500000	327.000000	

	availability_365
count	48895.000000
unique	NaN
top	NaN
freq	NaN
mean	112.781327
std	131.622289
min	0.000000
25%	0.000000
50%	45.000000
75%	227.000000

```
max           365.000000
```

1.5 4) Tratamento de Valores Faltantes

Valores faltantes são comuns em datasets reais e podem afetar a qualidade da análise. Nesta seção, identificamos e tratamos esses valores, decidindo entre remover, preencher com um valor específico (média, mediana, moda) ou usar métodos mais avançados.

[127]:
```
# Calcule e exiba a quantidade de valores faltantes por coluna
print("Valores faltantes por coluna:")
df.isna().sum()
```

```
Valores faltantes por coluna:
```

[127]:
```
id                                  0
name                               16
host_id                             0
host_name                          21
neighbourhood_group                 0
neighbourhood                       0
latitude                            0
longitude                           0
room_type                           0
price                               0
minimum_nights                      0
number_of_reviews                   0
last_review                     10052
reviews_per_month               10052
calculated_host_listings_count      0
availability_365                    0
dtype: int64
```

[128]:
```
# Preencha os valores faltantes nas colunas `name` e `host_name` com "unknown"
df["name"] = df["name"].fillna("unknown")
df["host_name"] = df["host_name"].fillna("unknown")

# Preencha os valores faltantes na coluna `reviews_per_month` com 0
df["reviews_per_month"] = df["reviews_per_month"].fillna(0)

# Converta a coluna `last_review` para datetime e preencha os valores faltantes␣
↪com uma data de referência (e.g., "2010-01-01")
df["last_review"] = pd.to_datetime(df["last_review"])
df["last_review"] = df["last_review"].fillna("2010-01-01")

# Verifique novamente os valores faltantes após o tratamento
print("Valores faltantes por coluna após tratamento:")
df.isna().sum()
```

```
Valores faltantes por coluna após tratamento:
```

[128]:
```
id                                0
name                              0
host_id                           0
host_name                         0
neighbourhood_group               0
neighbourhood                     0
latitude                          0
longitude                         0
room_type                         0
price                             0
minimum_nights                    0
number_of_reviews                 0
last_review                       0
reviews_per_month                 0
calculated_host_listings_count    0
availability_365                  0
dtype: int64
```

1.6 5) Detecção e Tratamento de Outliers

Outliers são pontos de dados que se desviam significativamente de outras observações. Eles podem distorcer análises estatísticas e modelos de machine learning. Nesta seção, identificamos e tratamos outliers em variáveis numéricas chave, como `price` e `minimum_nights`. Usaremos o método do Intervalo Interquartil (IQR) para uma abordagem robusta.

[129]:
```
# Defina uma função `remove_outliers_iqr` para detectar e remover outliers␣
↪usando o método IQR
def remove_outliers_iqr (df, column):

    q1 = df[column].quantile(0.25)
    q3 = df[column].quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr

    return df[(df[column] >= lower) & (df[column] <= upper)]
```

[130]:
```
print("Estatísticas antes do tratamento:")
print(df[["price", "minimum_nights", "number_of_reviews"]].describe())
print("\n")

# Aplique a função para tratar outliers nas colunas `price`, `minimum_nights` e␣
↪`number_of_reviews`
df = remove_outliers_iqr(df, "price")
df = remove_outliers_iqr(df, "minimum_nights")
df = remove_outliers_iqr(df, "number_of_reviews")
```

```
# Exiba as estatísticas descritivas antes e depois do tratamento para cada␣
↪coluna
print("Estatísticas após tratamento:")
print(df[["price", "minimum_nights", "number_of_reviews"]].describe())

# Atualize o DataFrame principal com os dados limpos
```

```
Estatísticas antes do tratamento:
              price  minimum_nights  number_of_reviews
count  48895.000000    48895.000000       48895.000000
mean     152.720687        7.029962          23.274466
std      240.154170       20.510550          44.550582
min        0.000000        1.000000           0.000000
25%       69.000000        1.000000           1.000000
50%      106.000000        3.000000           5.000000
75%      175.000000        5.000000          24.000000
max    10000.000000     1250.000000         629.000000


Estatísticas após tratamento:
              price  minimum_nights  number_of_reviews
count  35202.000000    35202.000000       35202.000000
mean     119.289643        2.765042          12.362536
std       67.831566        1.917473          16.631476
min        0.000000        1.000000           0.000000
25%       65.000000        1.000000           1.000000
50%      100.000000        2.000000           5.000000
75%      155.000000        3.000000          17.000000
max      334.000000       11.000000          71.000000
```

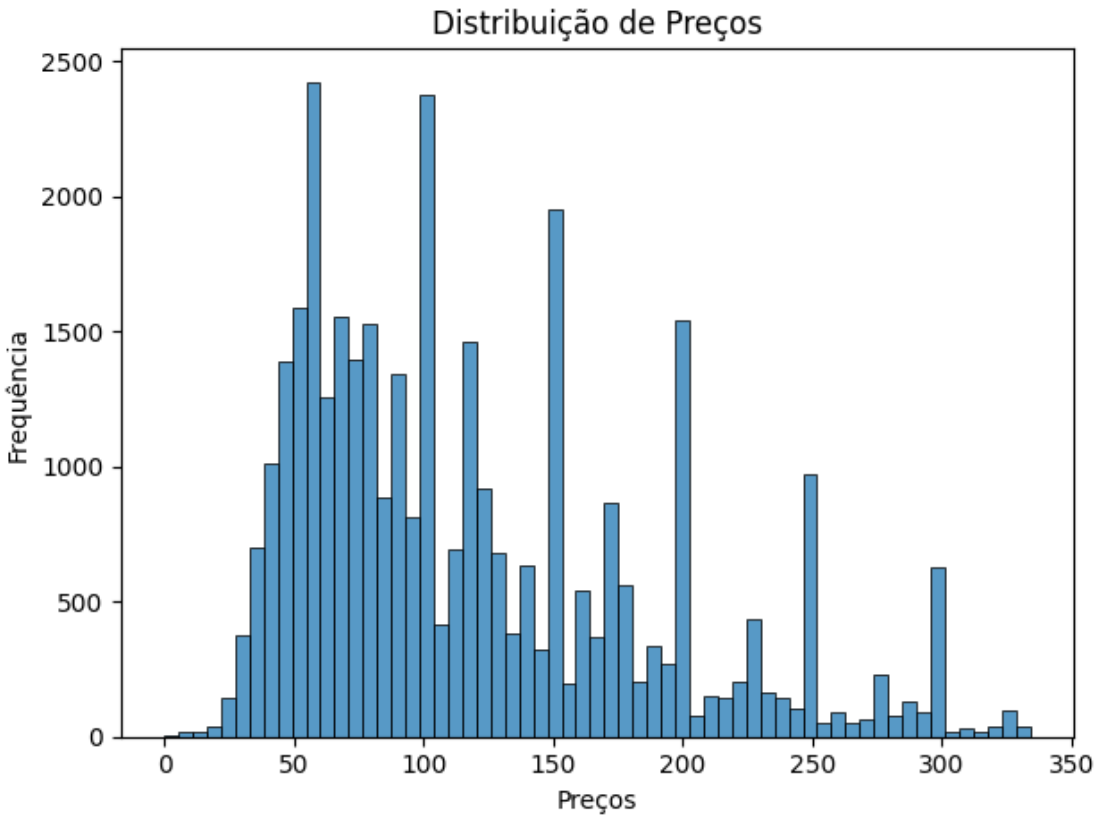
1.7 6) Análise Exploratória de Dados (EDA) Aprofundada

Com os dados limpos, podemos agora aprofundar nossa análise, explorando as distribuições das variáveis, as relações entre elas e identificando padrões que podem gerar insights valiosos.

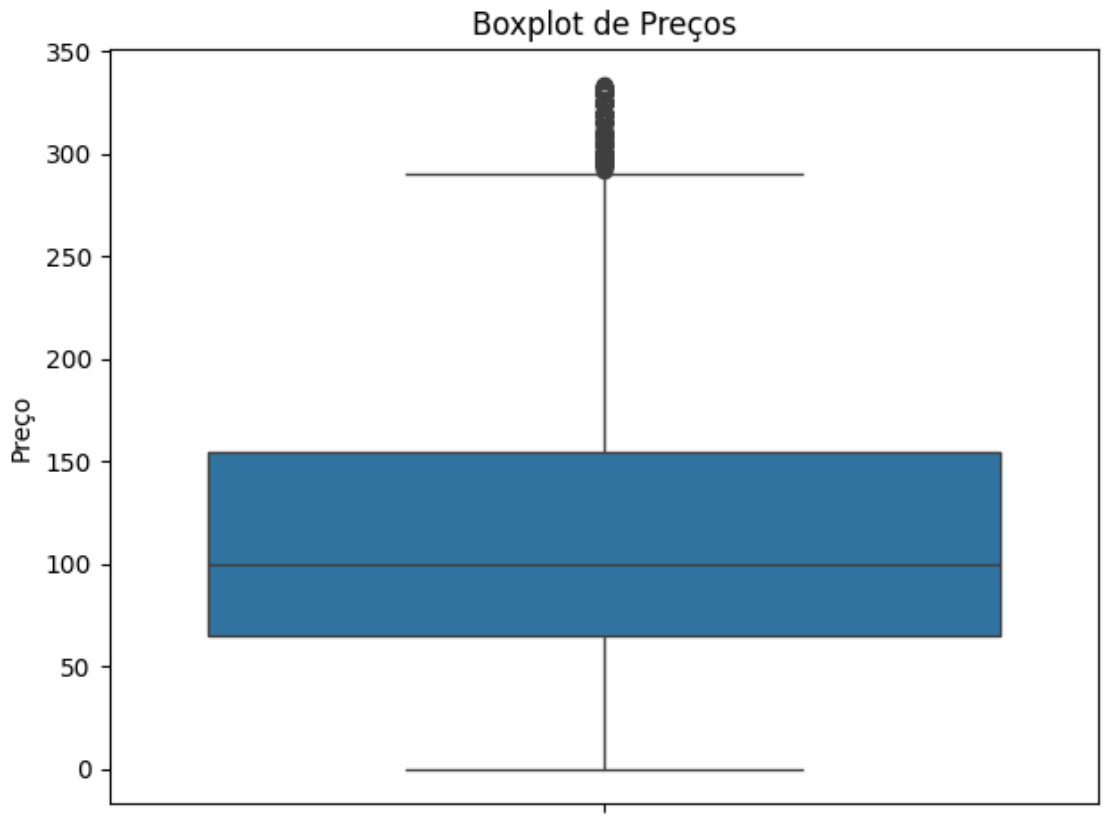
1.7.1 6.1) Distribuição de Preços

Vamos visualizar a distribuição dos preços dos aluguéis para entender a faixa de valores e identificar a concentração de preços.

[131]:
```
# Crie um histograma da coluna `price` usando `sns.histplot`
plt.figure()
sns.histplot(df["price"])
plt.title("Distribuição de Preços")
plt.ylabel("Frequência")
plt.xlabel("Preços")
plt.tight_layout()
plt.show()
```



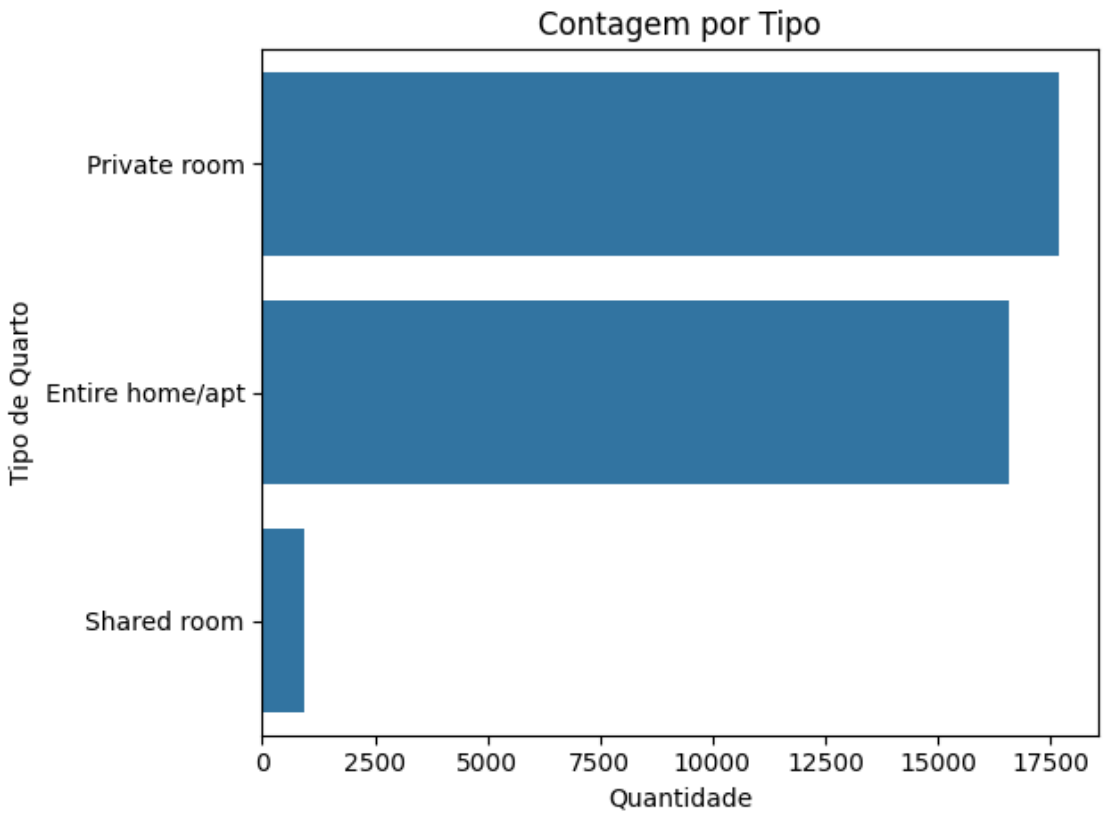
[132]:
```
# Crie um boxplot da coluna `price` usando `sns.boxplot`
plt.figure()
sns.boxplot(df["price"])
plt.title("Boxplot de Preços")
plt.ylabel("Preço")
plt.tight_layout()
plt.show()

# Adicione títulos e rótulos aos gráficos
```



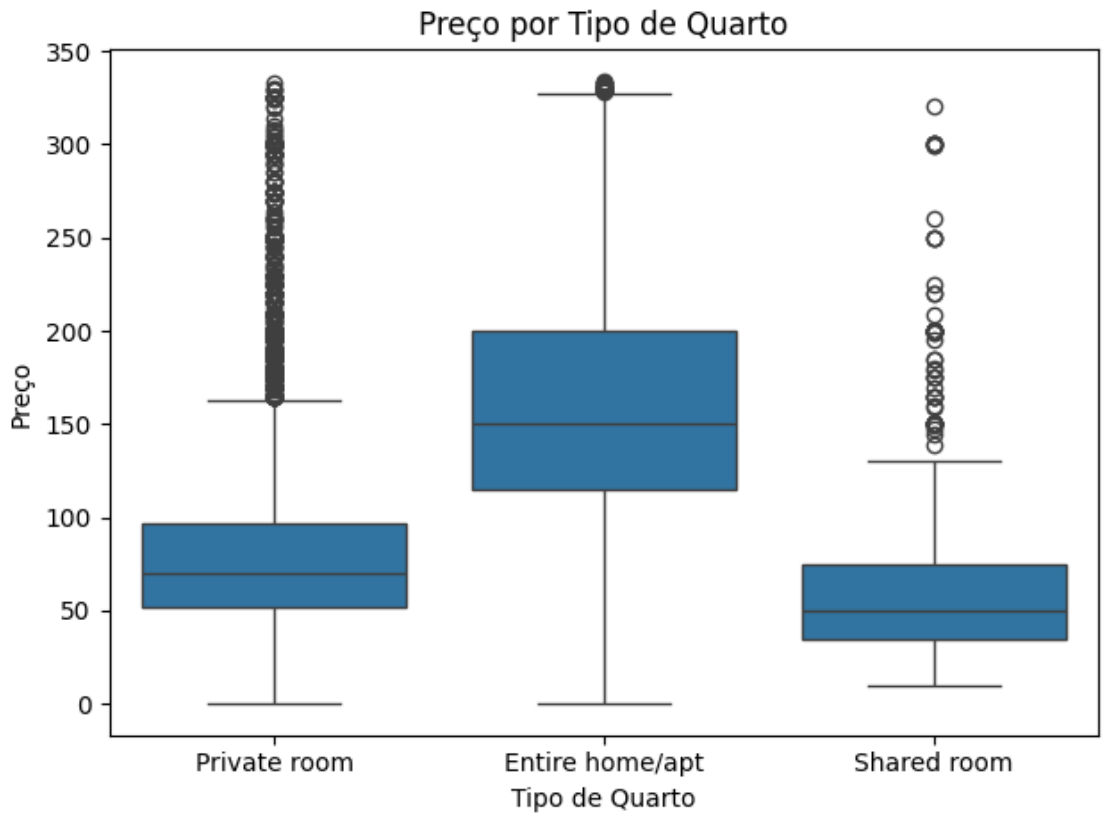
1.7.2 6.2) Análise por Tipo de Quarto (`room_type`)

Entender a distribuição dos tipos de quartos e como eles se relacionam com o preço é crucial para proprietários e hóspedes.

[133]:
```
# Crie um gráfico de contagem dos tipos de quarto usando `sns.countplot`
plt.figure()
sns.countplot(df["room_type"])
plt.title("Contagem por Tipo")
plt.xlabel("Quantidade")
plt.ylabel("Tipo de Quarto")
plt.tight_layout()
plt.show()
```



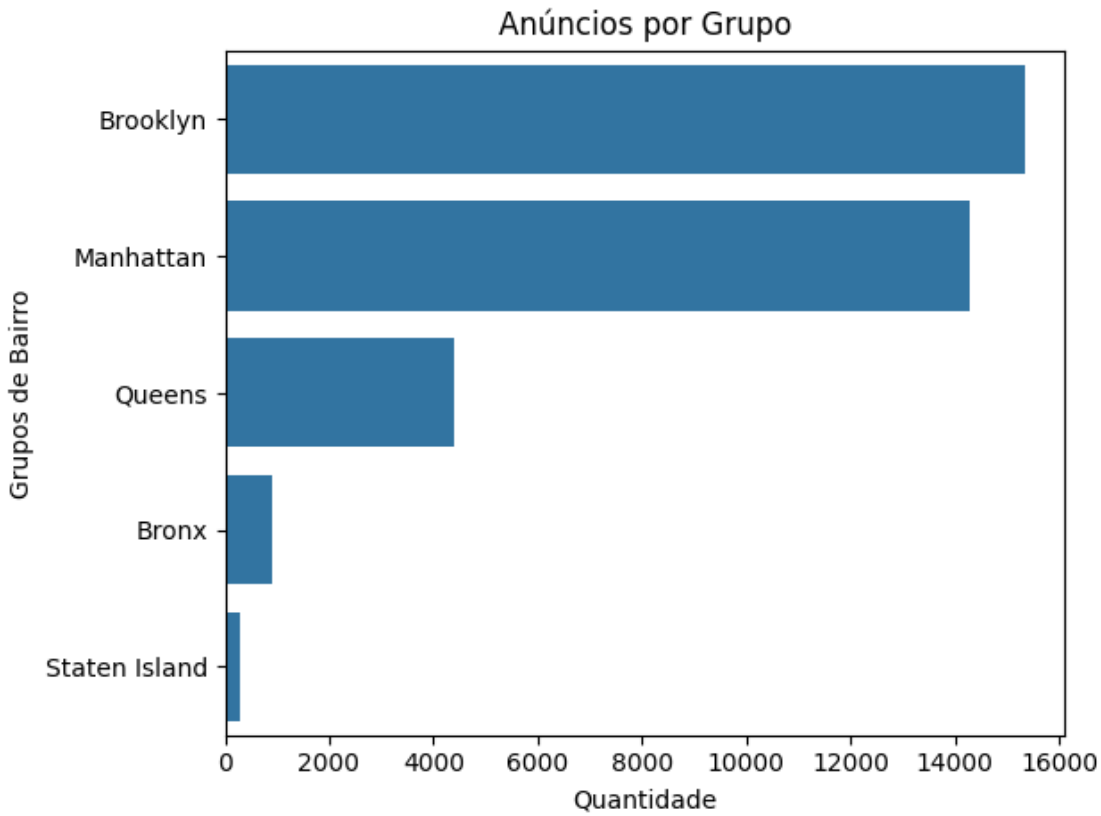
[134]:
```
# Crie um boxplot do preço por tipo de quarto usando `sns.boxplot`
plt.figure()
sns.boxplot(x = df["room_type"], y = df["price"])
plt.title("Preço por Tipo de Quarto")
plt.xlabel("Tipo de Quarto")
plt.ylabel("Preço")
plt.tight_layout()
plt.show()

# Adicione títulos e rótulos aos gráficos
```



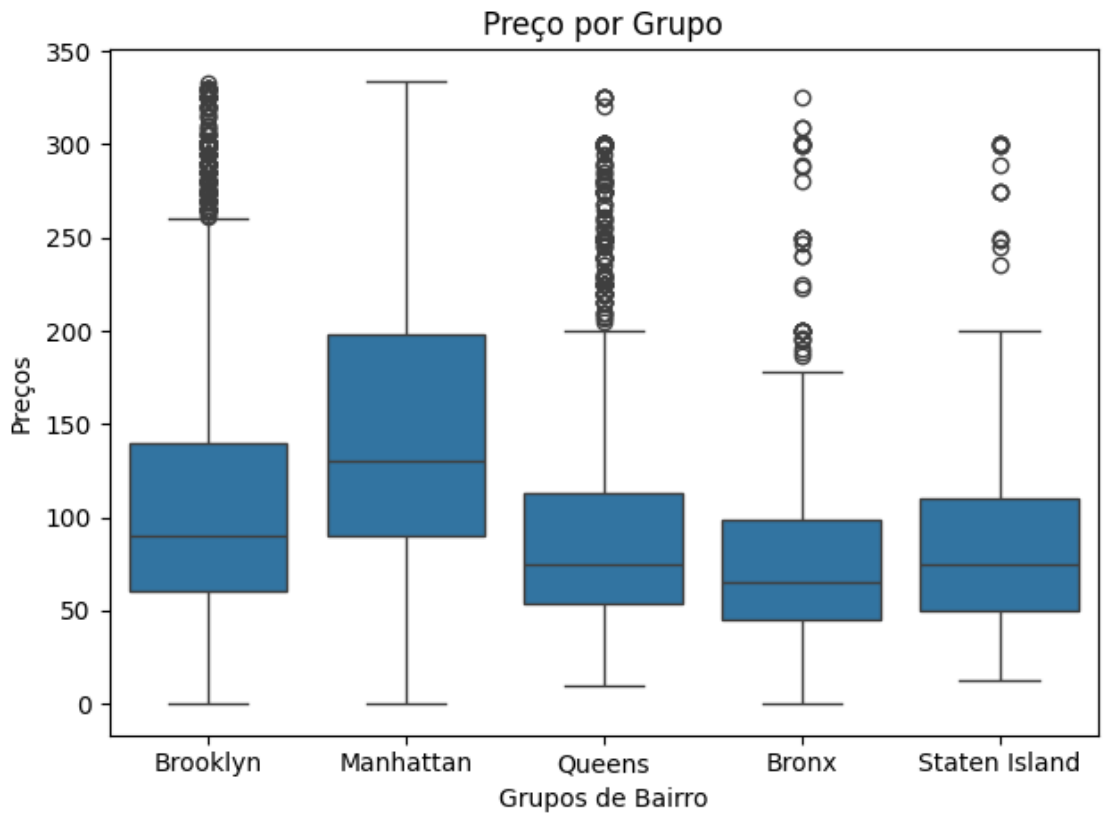
1.7.3 6.3) Análise por Grupo de Bairro (`neighbourhood_group`)

A localização é um dos fatores mais importantes no mercado imobiliário. Vamos analisar como os preços e a disponibilidade variam entre os diferentes grupos de bairros de NYC.

[135]:
```
# Crie um gráfico de contagem de anúncios por grupo de bairro usando `sns.
↪countplot`
plt.figure()
sns.countplot(df["neighbourhood_group"])
plt.title("Anúncios por Grupo")
plt.xlabel("Quantidade")
plt.ylabel("Grupos de Bairro")
plt.tight_layout()
plt.show()
```



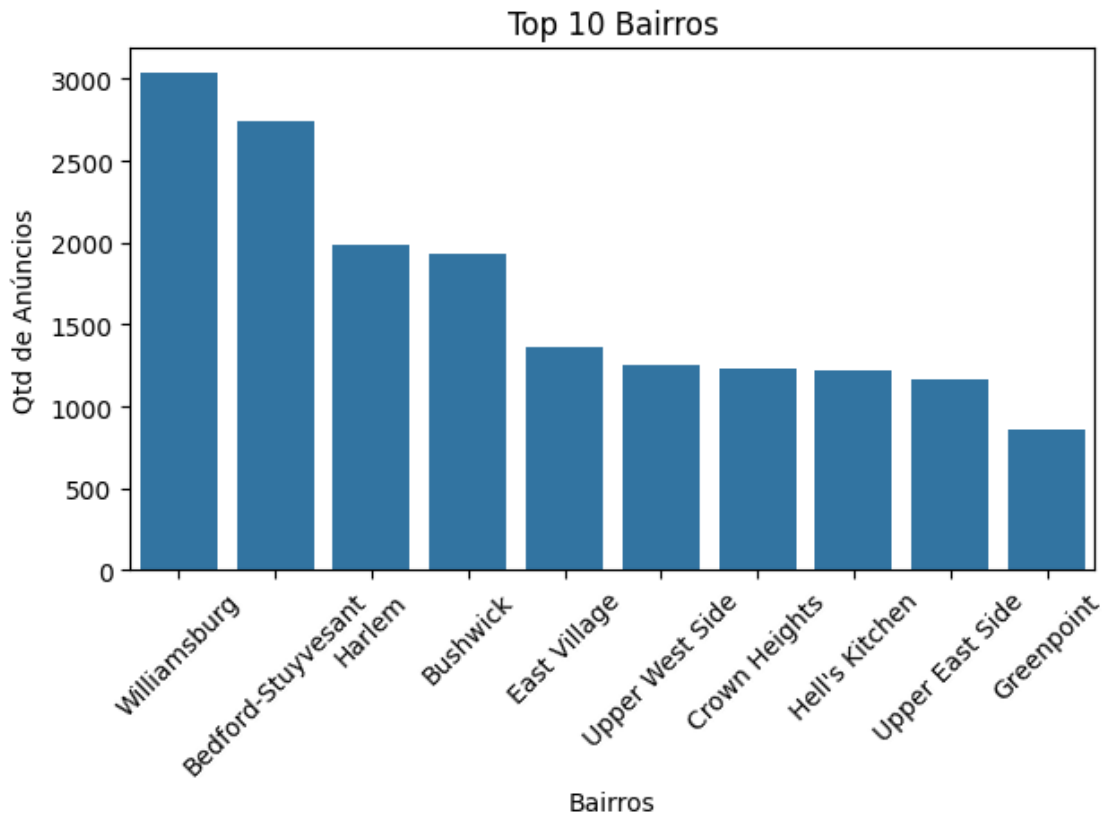
[136]:
```
# Crie um boxplot do preço por grupo de bairro usando `sns.boxplot`
plt.figure()
sns.boxplot(x = df["neighbourhood_group"], y = df["price"])
plt.title("Preço por Grupo")
plt.xlabel("Grupos de Bairro")
plt.ylabel("Preços")
plt.tight_layout()
plt.show()

# Adicione títulos e rótulos aos gráficos
```



1.7.4 6.4) Top 10 Bairros com Mais Anúncios

Identificar os bairros com maior concentração de anúncios pode indicar áreas de alta demanda ou popularidade.

[137]:
```
# Calcule os 10 bairros com mais anúncios
top_hood = df["neighbourhood"].value_counts().head(10)

# Crie um gráfico de barras para visualizar esses bairros usando `sns.barplot`
plt.figure()
sns.barplot(top_hood)
plt.title("Top 10 Bairros")
plt.xlabel("Bairros")
plt.ylabel("Qtd de Anúncios")
plt.xticks(rotation = 45)
plt.tight_layout()
plt.show()

# Adicione títulos e rótulos aos gráficos, e rotacione os rótulos do eixo X se␣
↪necessário
```



1.7.5 6.5) Análise Geoespacial dos Anúncios

Visualizar os anúncios em um mapa nos permite identificar clusters geográficos e entender a distribuição espacial dos imóveis.

[138]:
```
# Crie um mapa base de Nova York usando `folium.Map`
ny_map = folium.Map(location = [40.7128, -74.006], zoom_start = 10)

# Adicione um `MarkerCluster` ao mapa
from folium.plugins import MarkerCluster
marker_cluster = MarkerCluster().add_to(ny_map)

# Amostre uma parte do DataFrame (e.g., 5000 pontos) para melhor performance na␣
↪visualização
sample = df.sample(n = 5000)

# Adicione marcadores ao mapa para cada ponto amostrado, incluindo informações␣
↪no popup
for i, row in sample.iterrows():
    folium.Marker(
        location = [row["latitude"], row["longitude"]],
```

```
        popup = row["name"]
    ).add_to(marker_cluster)

# Salve o mapa em um arquivo HTML (e.g., "airbnb_nyc_map.html")
ny_map.save("airbnb_nyc_map.html")

# (Opcional) Exiba o mapa diretamente no notebook se o ambiente suportar
ny_map
```

[138]: <folium.folium.Map at 0x21cabbfc6b0>

1.8 7) Engenharia de Features

A engenharia de features envolve a criação de novas variáveis a partir das existentes para melhorar a capacidade preditiva dos modelos ou para obter novos insights. Aqui, criaremos algumas features simples.

1.8.1 7.1) `host_experience_days`

Calcularemos a experiência do anfitrião em dias, baseando-nos na data da última avaliação (`last_review`) e uma data de referência (ou a data da primeira avaliação, se disponível). Isso pode indicar a longevidade e experiência do anfitrião na plataforma.

[139]:
```
# Calcule a data mais recente de `last_review`
most_recent_date = df["last_review"].max()

# Crie a nova feature `host_experience_days` calculando a diferença em dias␣
↪entre a data mais recente e `last_review`
df["host_experience_days"] = most_recent_date - df["last_review"]

# Exiba as estatísticas descritivas da nova feature
df["host_experience_days"].dt.days.describe()
```

[139]:
```
count    35202.000000
mean       912.940032
std       1301.936154
min          0.000000
25%         22.000000
50%        190.000000
75%       1150.000000
max       3475.000000
Name: host_experience_days, dtype: float64
```

1.8.2 7.2) `price_per_night_per_review`

Esta feature pode indicar o custo-benefício percebido, relacionando o preço por noite com o número de avaliações. Um valor baixo pode indicar um bom negócio, enquanto um valor alto pode sugerir um preço elevado para poucos reviews.

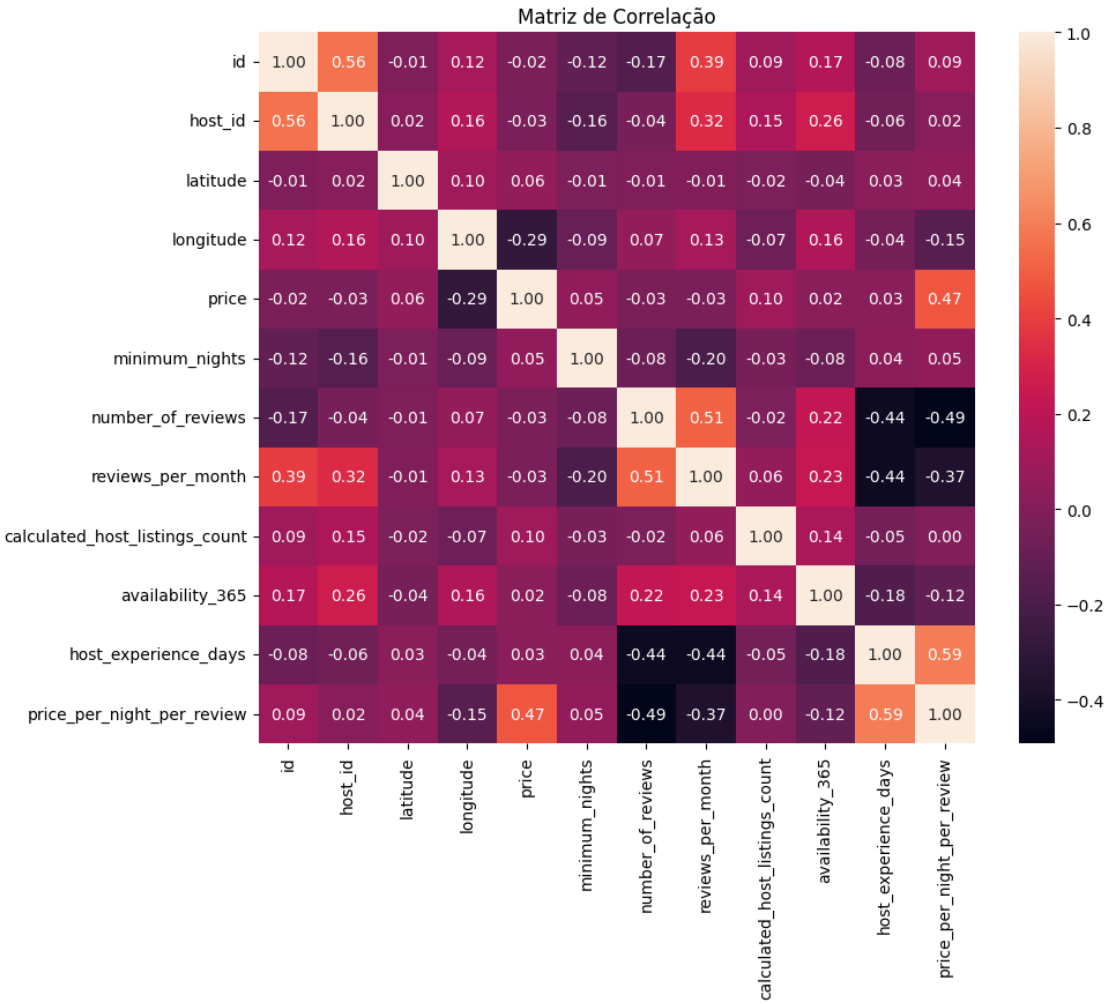
[140]:
```
# Crie a nova feature `price_per_night_per_review`, dividindo `price` por␣
↪`number_of_reviews`
# Lembre-se de tratar a divisão por zero (e.g., se `number_of_reviews` for 0,␣
↪use o próprio `price`)
df["price_per_night_per_review"] = df["price"] / df["number_of_reviews"].
↪replace(0, 1)

# Exiba as estatísticas descritivas da nova feature
df["price_per_night_per_review"].describe()
```

[140]:
```
count    35202.000000
mean        49.967902
std         63.791240
min          0.000000
25%          5.801613
50%         21.966667
75%         70.000000
max        333.000000
Name: price_per_night_per_review, dtype: float64
```

1.9 8) Análise de Correlação

A correlação mede a força e a direção da relação linear entre duas variáveis numéricas. Um mapa de calor (heatmap) da matriz de correlação é uma excelente ferramenta para visualizar essas relações.

[141]:
```
# Selecione apenas as colunas numéricas do DataFrame
num_rows = df.select_dtypes(include = "number")

# Calcule a matriz de correlação
corr_mat = num_rows.corr()

# Crie um heatmap da matriz de correlação usando `sns.heatmap`
plt.figure(figsize = (10,8))
sns.heatmap(corr_mat, annot = True, fmt = ".2f")
plt.title("Matriz de Correlação")
plt.show()

# Adicione anotações e um mapa de cores adequado
# Adicione um título ao gráfico
```


Matriz de Correlação
id 1.00 0.56 -0.01 0.12 -0.02 -0.12 -0.17 0.39 0.09 0.17 -0.08 0.09
host_id 0.56 1.00 0.02 0.16 -0.03 -0.16 -0.04 0.32 0.15 0.26 -0.06 0.02
latitude -0.01 0.02 1.00 0.10 0.06 -0.01 -0.01 -0.01 -0.02 -0.04 0.03 0.04
longitude 0.12 0.16 0.10 1.00 -0.29 -0.09 0.07 0.13 -0.07 0.16 -0.04 -0.15
price -0.02 -0.03 0.06 -0.29 1.00 0.05 -0.03 -0.03 0.10 0.02 0.03 0.47
minimum_nights -0.12 -0.16 -0.01 -0.09 0.05 1.00 -0.08 -0.20 -0.03 -0.08 0.04 0.05
number_of_reviews -0.17 -0.04 -0.01 0.07 -0.03 -0.08 1.00 0.51 -0.02 0.22 -0.44 -0.49
reviews_per_month 0.39 0.32 -0.01 0.13 -0.03 -0.20 0.51 1.00 0.06 0.23 -0.44 -0.37
calculated_host_listings_count 0.09 0.15 -0.02 -0.07 0.10 -0.03 -0.02 0.06 1.00 0.14 -0.05 0.00
availability_365 0.17 0.26 -0.04 0.16 0.02 -0.08 0.22 0.23 0.14 1.00 -0.18 -0.12
host_experience_days -0.08 -0.06 0.03 -0.04 0.03 0.04 -0.44 -0.44 -0.05 -0.18 1.00 0.59
price_per_night_per_review 0.09 0.02 0.04 -0.15 0.47 0.05 -0.49 -0.37 0.00 -0.12 0.59 1.00
id
host_id
latitude
longitude
price
minimum_nights
number_of_reviews
reviews_per_month
calculated_host_listings_count
availability_365
host_experience_days
price_per_night_per_review
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4